# Title: College Lost & Found Management System Using Singly Linked List, Stack and Queue

## 1. Problem Statement

To develop a College Lost & Found Management System using Singly Linked List, Stack, and Queue in C++. The system stores details of lost and found items such as Item ID, Item Name, and Status (Lost/Found). It allows users to add, search, display, and remove items using a linked list. Stack operations (Push and Pop) and Queue operations (Enqueue and Dequeue) are implemented using linked lists. The system also demonstrates dynamic memory allocation using the calloc() function.

## 2. Objectives

• To understand the concept of Singly Linked List, Stack, and Queue.

• To implement dynamic memory allocation as required by the problem statement.

• To add and store lost and found item details dynamically.

• To search for an item using its Item ID.

• To remove an item from the linked list.

• To display item details and their status efficiently.

• To implement Stack Push and Pop operations using a linked list.

• To implement Queue Enqueue and Dequeue operations using a linked list.

• To understand the practical application of data structures in college data management.

• To improve programming and problem-solving skills.

## 3. Algorithm

Step 1: Start the program.

Step 2: Define a Node structure containing Item ID, Item Name, Status, and a pointer named next.

Step 3: Initialize head, top, front, and rear pointers to NULL.

Step 4: Display a menu with options to add, search, remove, display items, perform Stack operations, perform Queue operations, and exit.

Step 5: To add an item, create a new node dynamically and accept its ID, name, and status.

Step 6: If the linked list is empty, make the new node the head; otherwise, traverse the list and add the node at the end.

Step 7: To search for an item, traverse the list using Item ID and display the details if the item is found.

Step 8: To remove an item, find its node, adjust the links, and release the node's memory.

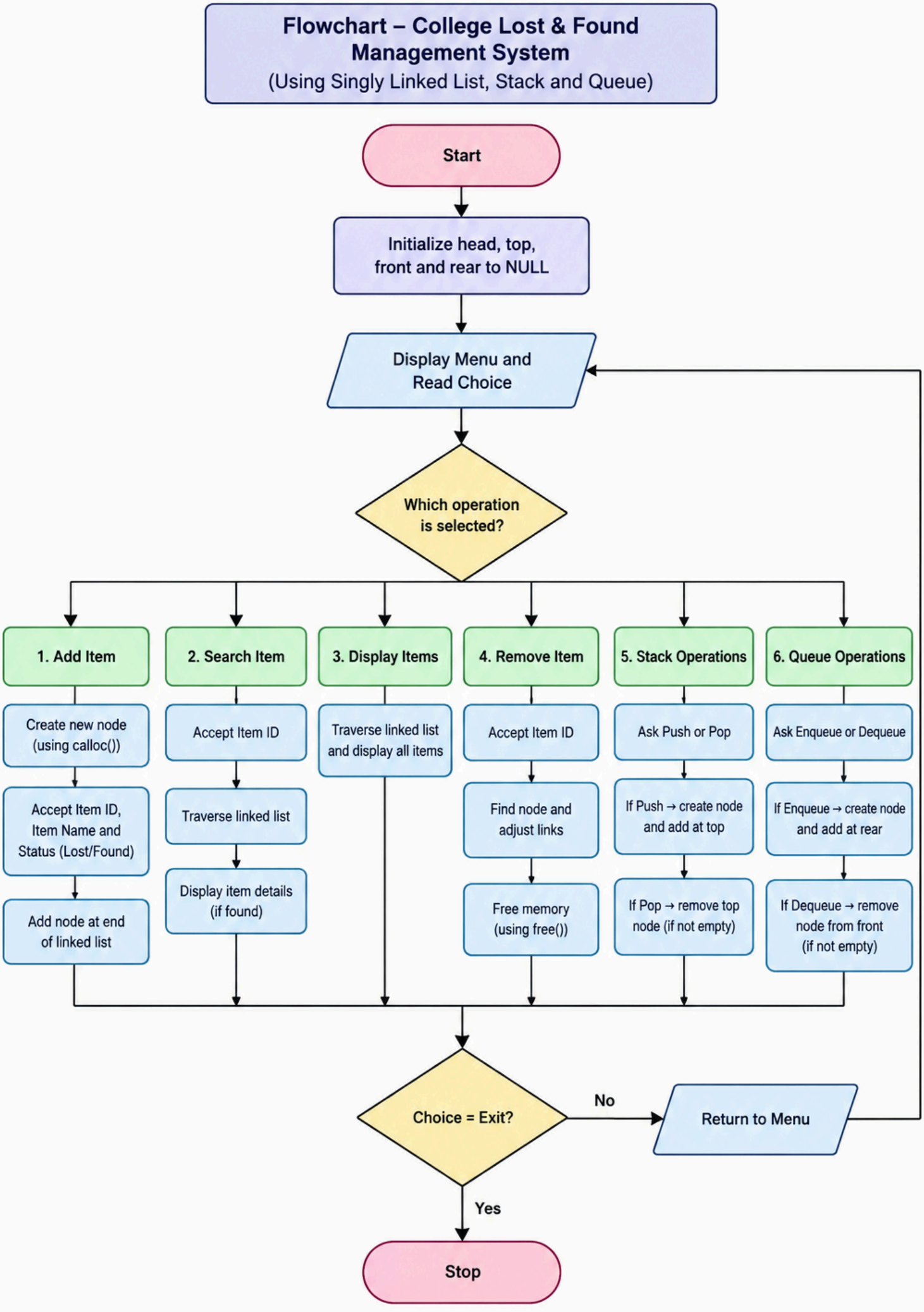
Step 9: For Stack Push, insert a node at top; for Stack Pop, remove the top node if the stack is not empty.

Step 10: For Queue Enqueue, insert a node at rear; for Queue Dequeue, remove a node from front if the queue is not empty.

Step 11: Use calloc() for dynamic memory allocation as required, ensuring that allocation and deallocation are compatible with the actual node structure.

Step 12: Repeat the menu until the user selects Exit.

Step 13: Stop the program.


Flowchart – College Lost & Found Management System
(Using Singly Linked List, Stack and Queue)
Start
Initialize head, top, front and rear to NULL
Display Menu and Read Choice
Which operation is selected?
1. Add Item
Create new node (using calloc())
Accept Item ID, Item Name and Status (Lost/Found)
Add node at end of linked list
2. Search Item
Accept Item ID
Traverse linked list
Display item details (if found)
3. Display Items
Traverse linked list and display all items
4. Remove Item
Accept Item ID
Find node and adjust links
Free memory (using free())
5. Stack Operations
Ask Push or Pop
If Push → create node and add at top
If Pop → remove top node (if not empty)
6. Queue Operations
Ask Enqueue or Dequeue
If Enqueue → create node and add at rear
If Dequeue → remove node from front (if not empty)
Choice = Exit?
No
Return to Menu
Yes
Stop

## 5. Conclusion

The College Lost & Found Management System using Singly Linked List, Stack, and Queue demonstrates the practical implementation of data structure concepts in C++. It provides a simple and dynamic method for adding, searching, removing, and displaying lost and found item records. Stack Push and Pop operations demonstrate the LIFO principle, while Queue Enqueue and Dequeue operations demonstrate the FIFO principle. The project also introduces dynamic memory allocation using calloc() as specified in the problem statement. Overall, it helps in understanding nodes, pointers, traversal, linked-list operations, stack, queue, and memory management.